

ARCHITECTURE

Scriba architecture documentation

Table of contents

1	Folder layout	2
2	Request flow	3
3	Patterns	3
3.1	Includes as data-fetch steps	3
3.2	POST-Redirect-Get (PRG) with session messages	3
3.3	Defensive null-guards in fetchers	3
3.4	Access levels	3
3.5	Database access	4
3.6	Output escaping	4
4	File and folder naming conventions	4
4.1	Folders	4
4.2	Files	4
5	File naming migration plan	5
5.1	Mapping	5
5.2	Variable-built and depth-dependent paths	7
5.3	Execution steps	7
6	Schema-name migration plan	8
6.1	Table mapping	8
6.2	Column mapping	9
6.3	Section values go lowercase	9
6.4	Application-level identifiers	9
6.5	TODO: changes beyond direct renaming	10
6.6	Execution steps	11
7	Database schema	11
7.1	Entity-relationship overview	11
7.2	Naming and conventions	12
7.3	Delete behavior	12
7.4	GDPR compliance	12
8	TODO: front-end / back-end implementation items	13

IMS (Inventory/Experiment Management System) is a plain PHP website with no framework. Every page is a PHP script that runs top-to-bottom and renders its own HTML. The structure below describes how files are organized and the patterns the code follows.

1 Folder layout

```
/          Page scripts (one PHP file per page)
├─ index.php      Login / start page
├─ experiment.php  Experiment overview page
├─ experiment_section.php  Experiment section page (plan/log/result via ?
section=)
├─ project_library.php    Listing pages, profile pages, etc.
|
├─ session/        Session handling
|   ├─ init.php      Starts the session (guarded, safe to include anywhere)
|   └─ check_user_logged_in.php
|
├─ database/        Database connection
|   ├─ db.php         Opens $conn (mysqli)
|   ├─ close_db.php   Closes $conn
|   └─ database_schema.sql  The MySQL schema (canonical)
|
├─ includes/        Reusable includes
|   ├─ check_user_permission.php  check_user_permission(): access level per
entity
|   ├─ navbar.php      Layout (included on every page)
|   ├─ fetch_project_experiment_names.php  Fetchers: included inline, expect
input variables ($experiment_id),
|   ├─ fetch_experiment_project_id.php      leave their result in a named
|   ├─ fetch_experiment_progress.php         variable ($exp_tags_array, ...)
|   ├─ fetch_experiment_tags.php
|   ├─ fetch_experiment_section_text.php
|   ├─ fetch_experiment_timestamps.php
|   └─ render_progress_badge.php  Helper function returning HTML for a progress
badge
|
├─ actions/         Write endpoints (PRG POST handlers, never render)
|   ├─ create_profile.php      Registration handler
|   ├─ logout.php              Destroys session, redirects
|   ├─ save_experiment_section.php  Section text/done-flag handler
|   └─ save_experiment_tags.php    Tag add/remove handler
|
└─ assets/           Static files
    ├─ style.css         Global stylesheet
    ├─ placeholder.avif  Logo shown in the navbar
    ├─ user_profile.css   (currently empty)
    └─ website_background.jpg
```

2 Request flow

Every page script follows the same order:

1. **Session:** include `session/init.php` (and the logged-in check where needed).
2. **Input:** read parameters from `$_GET` / `$_POST` with ?? defaults.
3. **Database:** include `database/db.php` to get `$conn`.
4. **Permission check:** `check_user_permission($conn, $_SESSION['user_id'], <type>, <ID>)` returns an access level: 0 = none, 1 = read, 2 = edit, 3 = owner. Pages block on `< 1`; write endpoints block on `< 2`.
5. **Fetch and render:** include `fetch_experiment_*.php` helpers to load data, then echo the HTML for the page.

3 Patterns

3.1 Includes as data-fetch steps

Fetcher files (`fetch_experiment_*.php`) are not functions. They are included inline, expect variables like `$exp_ID` to already exist, and leave their result in a named variable (`$exp_tags_array`, `$exp_progress_flags`, `$exp_section_text`, `$exp_last_update`). Each file documents the expected input and returned variable in its header comment.

3.2 POST-Redirect-Get (PRG) with session messages

Write endpoints (`actions/save_experiment_section.php`, `actions/save_experiment_tags.php`) never render a page. They: 1. Validate the POST input: required fields (e.g. `exp_ID`) get a graceful “missing” message + redirect if absent, and input-driven column names are checked against a whitelist (`Plan` / `Log` / `Result`). 2. Re-check permission server-side (`>= 2` for writes) — the endpoint never trusts values posted by the form. 3. Collect feedback strings in a local `$messages` array. 4. Store it in the session under a page-specific key (`$_SESSION['messages_save_experiment_section']`). 5. `header("Location: ...") + exit()` back to the referring page.

The target page retrieves the message array from the session, unsets it, and prints it. This keeps refresh/reload from resubmitting forms and gives the user simple success/error feedback.

3.3 Defensive null-guards in fetchers

Fetchers handle the two “no data” cases explicitly: a query that matches no row (`fetch_assoc()` returns `null`) and a row whose column is `NULL`. Text fetchers default to `' '` (empty textarea for a new experiment); the timestamp fetcher defaults to `[]` and formats values with `date()`, so a missing row degrades to blank output rather than a fatal error. In practice `check_user_permission()` throws first for invalid IDs, so these guards are defensive.

3.4 Access levels

Access levels come from one central query in `user_permission.php` that walks the membership hierarchy (company → lab group → project → experiment) and returns the highest applicable level. Invalid or unknown IDs make the function throw `RuntimeException`; unimplemented entity

types ('project', 'lab', ...) return 0 with a PHP warning. UI adapts to the level: view-only users get static text, users with level ≥ 2 get the edit form.

3.5 Database access

All queries use **mysqli prepared statements**. Values are always bound with `bind_param()`. Column names that depend on input (e.g. the section name `Plan/Log/Result`) are never taken raw from the request: they are checked against a whitelist (`$valid_sections`) before being used in SQL.

3.6 Output escaping

Anything echoed into HTML goes through `htmlspecialchars()` to prevent XSS (this includes hidden input values, textarea content, and badge labels). `urlencode()` is used when building query strings in links/redirects.

4 File and folder naming conventions

The file system follows the same philosophy as the database schema (Database naming standards): lowercase snake_case everywhere, no abbreviations, names describe *what the file is*, with verbs reserved for endpoints that *do* something.

4.1 Folders

Folder names are lowercase, singular-free role names that say what the folder is for — not what technology is inside:

Folder	Role
/ (root)	Page scripts (one PHP file per page)
<code>session/</code>	Session bootstrap and auth guards
<code>database/</code>	DB connection helpers and the schema file
<code>includes/</code>	Reusable includes: functions, layout, data fetchers
<code>actions/</code>	Write endpoints (PRG POST handlers that redirect)
<code>assets/</code>	Static files: CSS, images
<code>docs/</code>	Markdown documentation

4.2 Files

- **Pages** (root): noun, named after what the user sees — `experiment_section.php`, `leaderboard.php`.
- **Write endpoints** (`actions/`): verb first, describing the state change — `save_experiment_section.php`, `create_profile.php`. Never render HTML.
- **Fetchers** (`includes/`): `fetch_<entity>_<what>.php` — `fetch_experiment_tags.php`. Included inline; each documents its expected input variable and the result variable it sets.
- **Function files** (`includes/`): one file per function, named after the function it defines — `render_progress_badge.php` defines `render_progress_badge()`.

- **Assets:** no double extensions; the extension states the actual format (`placeholder.avif`, not `placeholder.jpg.avif`).
- **No abbreviations:** `exp` → `experiment`, `proj` → `project`, `fun` → nothing (the file is named after its function). IDs are lowercase: `project_id`, never `proj_ID`.
- Schema and code share one vocabulary: files and variables use the same snake_case names as the database (`experiment_id`, `plan_text`), so the mapping between code and schema is obvious.

5 File naming migration plan

The codebase predates these conventions. The renames below were **executed on 2026-09-24** as a pure rename pass — file/folder names and `include` paths only. It is independent of (and can be done before or after) the schema migration listed in the TODO section, which changes names *inside* the files.

5.1 Mapping

Current	Target	Reason
<code>Session/</code>	<code>session/</code>	lowercase role name
<code>Session/init.php</code>	<code>session/init.php</code>	folder only
<code>Session/ check_user_logged_in.php</code>	<code>session/ check_user_logged_in.php</code>	folder only
<code>Database_related/</code>	<code>database/</code>	role name, no <code>_related</code> suffix
<code>Database_related/db.php</code>	<code>database/db.php</code>	folder only
<code>Database_related/ closeDB.php</code>	<code>database/close_db.php</code>	camelCase → snake_case
<code>Database_related/ database_schema.sql</code>	<code>database/ database_schema.sql</code>	folder only
<code>Functional_php/</code>	<code>includes/</code>	role name, no <code>_php</code> suffix
<code>Functional_php/ user_permission.php</code>	<code>includes/check_user_permission.php</code>	named after the function it defines
<code>Functional_php/ navbar.php</code>	<code>includes/navbar.php</code>	folder only
<code>Functional_php/style.css</code>	<code>assets/style.css</code>	it is an asset
<code>Functional_php/ logout.php</code>	<code>actions/logout.php</code>	write endpoint (destroys session, redirects)
<code>Functional_php/ exp_helpers/</code>	<code>includes/</code> (fetchers), <code>actions/</code> (endpoints)	split by role
<code>Functional_php/ exp_helpers/ exp_fetch_name.php</code>	<code>includes/fetch_project_experiment_names.php</code>	it fetches both names

Current	Target	Reason
Functional_php/ exp_helpers/ exp_fetch_proj_ID.php	includes/fetch_experiment_projects.php	no abbreviation, lowercase id
Functional_php/ exp_helpers/ exp_fetch_progress.php	includes/fetch_experiment_progress.php	spell out experiment
Functional_php/ exp_helpers/ exp_fetch_tags.php	includes/fetch_experiment_tags.php	spell out experiment
Functional_php/ exp_helpers/ exp_fetch_text.php	includes/fetch_experiment_section_text.php	it fetches one section's text
Functional_php/ exp_helpers/ exp_fetch_updated.php	includes/fetch_experiment_times.php	it actually returns times that it has
Functional_php/ exp_helpers/ exp_progress_fun.php	includes/render_progress_bar.php	named after the function, no fun
Functional_php/ exp_helpers/ exp_edit_section.php	actions/save_experiment_section.php	verb-first write endpoint
Functional_php/ exp_helpers/ exp_edit_tags.php	actions/ save_experiment_tags.php	verb-first write endpoint
register_user_page.php	register_user.php	drop redundant _page
insert_new_user.php	actions/ create_profile.php	verb-first, matches schema (profiles)
(root)placeholder.jpg.avif	assets/placeholder.avif	single, correct extension (assets lived at the root)
(root)user_profile.css	assets/user_profile.css	empty file; either remove it or give it content when profile styling is implemented
(root) website_background.jpg	assets/ website_background.jpg	move into assets/
ARCHITECTURE.md, NEWS.md, README.md	docs/ (optional)	only if the root should hold nothing but pages

Root page scripts (index.php, project_library.php, experiment.php, experiment_section.php, experiment_library.php, leaderboard.php, user_profile.php, company_admin.php, scriba_admin.php) already follow the conventions and stay put.

5.2 Variable-built and depth-dependent paths

Not every reference is a literal string. The following cases fall outside a plain find-and-replace and must be handled explicitly:

- **Section-page references (now consolidated):** the former `experiment_<section>.php` pages were removed in favor of the single `experiment_section.php?section=...` page. `exp_edit_section.php` previously built redirect URLs dynamically from the section name; its redirects now point at `experiment_section.php` with an explicit `section` parameter. No page-name mapping is needed for section pages, but the `section` parameter's values (`Plan/Log/Result`) will change to lowercase in the schema migration (`plan/log/result`) — the whitelist and the links in `experiment.php` must be updated together.
- **Folder-depth prefixes:** the write endpoints live two levels deep (`Functional_php/exp_helpers/`) and use `../..` in includes and redirects. Moving them to `actions/` (one level) changes every `../..` prefix to `../`, and their same-directory includes (`exp_fetch_progress.php`, `exp_fetch_tags.php`) break because those fetchers move to `includes/` — they become `../includes/fetch_experiment_progress.php`, etc. Same-directory includes are invisible to a path-string grep, so audit each moved file's `include` lines individually.
- **Case-inconsistent paths:** several files already include `database_related/db.php` / `functional_php/navbar.php` in lowercase. This works on Windows (case-insensitive filesystem) but breaks on Linux, so the reference search must be **case-insensitive**, and the rename pass is the right moment to normalize every path to the canonical spelling.
- **../ root-relative links:** `navbar.php` uses `href="/../project_library.php"` (etc.) and `logout.php` redirects to `../index.php`. These are fragile even today; fix them to proper relative paths as part of the same pass.
- **Commented-out links:** `exp_edit_tags.php` contains commented-out `href/redirect` code referencing old paths — update or delete it so a future un-commenting can't resurrect dead paths.
- **Posts-to-self forms** (`action=""` in `index.php`, `project_library.php`, `experiment_library.php`, `register_user_page.php`, `scriba_admin.php`, `company_admin.php`) are unaffected by renames, but note that `index.php` and `register_user_page.php` carry a "change action so you end up somewhere!" comment — their POST handling should be routed to an `actions/` endpoint when that work happens.

5.3 Execution steps

1. **Rename with `git mv`** so history follows the files. Order: folders first (`Session/`, `Database_related/`, `Functional_php/`, `Assets/`), then the files inside them (including the `exp_helpers/` split into `includes/` vs `actions/`).
2. **Update references in the same commit:** every `include/require` path, navbar `hrefs` and `<link>/src` URLs (navbar references the stylesheet and logo), form `action` attributes, and redirect targets in the PRG endpoints. Grep the whole tree for each old

path segment (Session/, Functional_php/, exp_helpers/, Database_related/, closeDB, insert_new_user, .jpg.avif) to confirm none remain.

3. **Update this document:** the folder-layout tree and any path mentions.
4. **Manual test sweep:** login → project library → experiment → section edit → tag add/remove → logout, plus registration. All navigation, POST targets, and assets must load without 404s.
5. **Commit as a single pure-rename commit**, separate from any schema migration, so `git log --follow` and review stay clean.

6 Schema-name migration plan

The PHP code queries the **old** schema. The mapping below renames every table and column reference to the new snake_case schema. Like the file renames, this is **planned but not yet executed**. It is a separate pass from the file-naming migration: do that one first (pure renames), then this one, so each commit diffs cleanly.

6.1 Table mapping

Old table	New table	Used in
Proj_Experiment (also misspelled Proj_Experiments in exp_fetch_progress.php)	experiments	fetch-ers, exp_edit_section.php, user_permission.php
Project	projects	exp_fetch_name.php, user_permission.php
Exp_Tag	experiment_tags	exp_fetch_tags.php, exp_edit_tags.php
User	profiles	registration, admin pages, user_permission.php
Company	companies	admin pages, user_permission.php
Lab_Group	labs	admin pages, user_permission.php
Company_Member	company_members	admin pages, user_permission.php
Lab_Group_Member	lab_members	admin pages, user_permission.php
Project_Member	project_members	user_permission.php
Experiment_Member	experiment_members	user_permission.php
Scriba_Member	(dissolved) — query profiles.is_scriba_admin = TRUE instead	user_permission.php

The last row is **TODO: a rewrite, not a rename**: the Scriba-admin check becomes `SELECT is_scriba_admin FROM profiles WHERE profile_id = ?` (level 3 if TRUE, 0 otherwise).

6.2 Column mapping

Old column	New column
Exp_ID / Experiment_ID (both spellings occur)	experiment_id
Proj_ID / Project_ID	project_id
User_ID	profile_id
Exp_Name / Proj_Name / Lab_Name / Comp_Name	name
Email, First_Name, Last_Name, Salt, Password	email, first_name, last_name, salt, password
Experiment_ID (in Exp_Tag)	experiment_id
Exp_Tag (column)	tag
Plan_Text / Log_Text / Result_Text	plan_text / log_text / result_text
Plan_Done / Log_Done / Result_Done	plan_is_done / log_is_done / result_is_done
Plan_Updated / Log_Updated / Result_Updated	plan_updated_at / log_updated_at / result_updated_at
Date_Created / Date_Updated	created_at / updated_at

6.3 Section values go lowercase

The section whitelist and query-parameter values change from Plan/Log/Result to plan/log/result, which changes every string-concatenated column name built from `$exp_section`:

- `$valid_sections` in `experiment_section.php` and `exp_edit_section.php`.
- Column interpolation: "... SET " . `$exp_section` . "_Text" becomes `$exp_section` . '_text'; likewise `_Done` → `_is_done` and `_Updated` → `_updated_at`.
- Array keys from `fetch_assoc()` (`$exp_progress_flags[$exp_section . '_Done']`, `$exp_last_update[...]`) follow the new column names.
- Links in `experiment.php` (§ion=Plan) and the redirects in `exp_edit_section.php`.
- Display labels may stay capitalized ("Experiment Plan"); only the machine values change.

6.4 Application-level identifiers

Code should share the schema's vocabulary, so variables and session keys migrate too:

Old	New
<code>\$_SESSION['user_id']</code> (set at login, checked in <code>check_user_logged_in.php</code>)	<code>\$_SESSION['profile_id']</code>

Old	New
<code>\$exp_ID</code>	<code>\$experiment_id</code>
<code>\$proj_ID</code>	<code>\$project_id</code>
<code>\$user_ID</code> parameter	of <code>\$profile_id</code>
<code>check_user_permission()</code>	
<code>\$proj_exp_name_array</code> keys <code>Project_Name,</code> <code>Experiment_Name, Proj_ID, Exp_ID</code>	<code>project_name,</code> <code>experiment_name,</code> <code>project_id, experiment_id</code>
<code>\$exp_last_update</code> keys <code>Date_Created,</code> <code>Date_Updated, Plan_Updated, ...</code>	<code>created_at, updated_at, plan_updated_at,</code> <code>...</code>
<code>\$exp_progress_flags</code> keys <code>Plan_Done, ...</code>	<code>plan_is_done, ...</code>

6.5 TODO: changes beyond direct renaming

Everything below is a code change over and above substituting new names. Each item must be marked with a `// TODO(schema-migration): ...` comment in the code during the pass, so the changes stay reviewable and greppable.

- **TODO — delete the manual timestamp update:** `exp_edit_section.php` runs `UPDATE ... SET <section>_Updated = NOW()`. In the new schema the `*_updated_at` columns have `ON UPDATE CURRENT_TIMESTAMP` (they update automatically when the row's text changes) and `experiments.updated_at` is a **generated column** — writing it raises an error. The whole timestamp-update statement goes away.
- **TODO — rework `exp_fetch_updated.php`:** it should not select `updated_at` for display via the generated column — it derives from the `*_updated_at` columns and defaults to the epoch sentinel (1970-01-01) when none exist; prefer showing `created_at` and the section timestamps.
- **TODO — fix `exp_fetch_progress.php`:** it has a typo table name (`Proj_Experiments`) and fails against both old and new schemas; this pass fixes it, which is a fix, not a rename.
- **TODO — fix `exp_fetch_proj_ID.php`:** it selects `Project_ID` while the same query's `WHERE` uses `Experiment_ID` and the join keys use `Proj_ID/Exp_ID` — the old code is internally inconsistent; resolving that is a fix beyond the rename.
- **`check_user_permission()`:** the entity-type strings (`'experiment', 'project', 'lab', 'company'`) already match the new schema and stay; only the table/column names in the five queries change (pure rename).
- **TODO — admin pages:** `company_admin.php` and `scriba_admin.php` run against the old tables; beyond the table/column mapping, the `company_admin.php` `$admin_ID = ""` placeholder must be resolved from the session (`$_SESSION['profile_id']`).
- **TODO — registration:** `insert_new_user.php` and `register_user_page.php` (which duplicate the same query) need the new `profiles` columns (`agreed_to_toc`, and defaults for `saved_changes / streak` — decide the values at registration time), and should set `last_login_at` or write a `login_log` row per the TODO items.
- **TODO — mock-data pages:** `leaderboard.php`, `project_library.php`, `experiment_library.php` use hardcoded arrays with keys `id/name`; they need no rename but

will adopt the schema vocabulary when converted to real queries (the `profile_points` and `project_updates` views are designed for them).

- **TODO** — `db.php`: the database name must match the schema-loaded database.

6.6 Execution steps

1. Run **after** the file-naming migration, in its own commit(s).
2. Update each file's queries, interpolated column names, whitelist, and array keys per the mapping above; keep the display copy capitalized as needed. Each spot needing change is already marked in the code with a `// TODO(schema-migration): ...` comment stating what it becomes — resolve each marker while updating the names, so the comments can be grepped to track remaining work.
3. Load the new schema into a fresh database and adjust `db.php`.
4. Grep case-insensitively for every old identifier (`Proj_`, `Exp_`, `User_ID`, `Company_Member`, `Lab_Group`, `Scriba_Member`, `_Done`, `_Updated`, `Date_`, `user_id`) — none may remain outside comments/history.
5. Test sweep: register → login → library → experiment → section edit (text + done flag, verify timestamps auto-update) → tag add/remove → logout; plus a permission check for a read-only user.

7 Database schema

The canonical schema lives in `database/database_schema.sql` (MySQL). It defines the core entity tables (`companies`, `labs`, `projects`, `experiments`, `profiles`), membership/tag junction tables, an audit layer (`activity_log`, `login_log`), and two views (`project_updates`, `profile_points`).

7.1 Entity-relationship overview

```
companies — labs — projects — experiments

profiles —┐ company_members — companies
           │ lab_members — labs
           │ project_members — projects
           └─ experiment_members — experiments

projects — project_tags      (project_id, tag)
experiments — experiment_tags (experiment_id, tag)

profiles — activity_log      (who did what, entity_type + entity_id)
profiles — login_log         (nullable: failed logins have no profile)

views: project_updates (project + latest experiment update),
       profile_points  (points per profile, excludes deleted profiles)
```

Containment is strict: a company holds labs, a lab holds projects, a project holds experiments. Memberships connect a profile to exactly one entity each, with a per-level *role* (owner/edit/read on projects and experiments, admin/member on labs and companies).

7.2 Naming and conventions

The schema follows Database naming standards:

- snake_case everywhere; plural table names, singular column names.
- Generic column names per table: `name`, `created_at`, `updated_at`, `is_done`.
- Content tables use `created_at/updated_at` (row lifecycle); log tables name the timestamp after the event instead — `login_log.login_at`, `activity_log.acted_at` — since the row's creation is the event.
- FKs are named after the PK they reference (`profile_id` in every table).
- PKs are auto-increment `INT`, except `company_id/lab_id` which are prefixed `VARCHAR(10)` (`c1`, `l1`, ...) to avoid collisions.
- `*_updated_at` columns are `DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP`; `is_done` flags default to `FALSE`.
- `experiments.updated_at` and `experiments.is_done` are **stored generated columns** derived from the plan/log/result sections, so an experiment's status can never disagree with its sections. `experiments.updated_at` is `DATETIME` (MySQL does not allow generated `TIMESTAMP` columns).

7.3 Delete behavior

- Deleting a company with labs, or a lab with projects, is `RESTRICTed` — handle in application code, in explicit order, in one transaction.
- Deleting a project cascades to its experiments, memberships, and tags. The UI must show a strong confirmation before deleting a project with experiments.
- Membership/tag junctions cascade on deletion of either side.

7.4 GDPR compliance

Profile “deletion” is **anonymization, never a hard delete**:

1. Set `is_deleted = TRUE` (login checks this flag *before* password verification).
2. Overwrite: `email` → `deleted_<profile_id>@example.com`, `names` → `Deleted`, `salt` → fresh random bytes, `password` → a random, discarded hash.
3. Anonymize the user's email in `login_log` rows (including failed logins, matched by email).
4. Memberships are **intentionally kept** — they reference the anonymized profile, preserving authorship/audit history without personal data.
5. Ownership transfer for projects whose only owner is deleted is handled by the company/lab admin.

Retention: `activity_log` and `login_log` are purged daily after 90 days (scheduled job). `profile_points` excludes deleted profiles (`WHERE is_deleted = FALSE`).

Deleted-profile display rule: **hide** where they would be actionable (login, member pickers/search); **render as “Deleted”** where they are historical (member lists, activity timelines, admin ownership views).

8 TODO: front-end / back-end implementation items

- ☒ **File naming migration:** executed 2026-09-24 — folders, files, and all include/link references updated in one pure-rename commit (see the File naming migration plan section).
- ☐ `anonymize_profile($profile_id)` — implements the GDPR flow above.
- ☐ `log_activity(...)` helper — writes `activity_log` rows in the same transaction as the change they describe (the DB cannot know the actor).
- ☐ Daily cron job purging `activity_log` / `login_log` rows older than 90 days.
- ☐ Login flow: check `is_deleted` before password verification; use `login_log` (email + 15-minute window) for rate limiting / lockout.
- ☐ Admin page: display project ownership, flagging projects owned by deleted profiles so admins can transfer them.
- ☐ Member management UI: exclude deleted profiles from pickers; show them as “Deleted” in existing member lists.
- ☐ Project deletion UI: strong confirmation when the project contains experiments.
- ☐ **Schema-name migration:** execute the plan in the Schema-name migration plan section — tables, columns, lowercase section values, application identifiers, and the behavioral changes (drop the manual timestamp update, fix the broken table names) — after the file-naming migration.